Universidad Autónoma de Chile
Base de Datos 2 - Unidad 2

Solemne 02
Diseño e implementación de una solución NoSQL para el monitoreo de espacios inteligentes

Caso 2026: Sistema de monitoreo inteligente de espacios universitarios

Campo	Dato
Nombre de grupo	La última base
Integrantes	Ignacio Ampuero, Matías Catalán, Felipe González, Iván Herrera, Benjamín Lagos, Adrián Mancilla Carter
Profesor	René Galarce Godoy
Fecha	08-11-2026

Índice

Introducción 2
Desafíos 3
Objetivo general 3
Fundamentación teórica 3
Comparación entre el enfoque relacional y NoSQL 4
Diseño del Esquema de Datos 4
Descripción del modelo propuesto 5
Colecciones principales 5
Relaciones entre documentos y cardinalidades relevantes 6
Justificación técnica de las decisiones de embebido y referencias 6
Representación JSON 7
Explicación del modelo 11
Diagrama del modelo 12
Inserción de Datos y Operaciones CRUD 12
Evidencia de código 13
4.2.1 Consultas de lectura 16
Consultas Avanzadas 16
Filtros 16
Operadores 16
Agregaciones 17
Discusión de Resultados 19
Verificación de ejecución en MongoDB Atlas (pruebas integrales) 19
Evaluación crítica del modelo implementado 22
Comparación con una alternativa relacional 23
Escalabilidad, flexibilidad del esquema y eficiencia de las consultas 23
Conclusiones 24
Bibliografía 26
Anexos 27

Introducción

La universidad ha iniciado un proyecto de modernización tecnológica enfocado en la gestión inteligente de sus espacios académicos. Para ello, se han desplegado dispositivos IoT en salas de clases, laboratorios y áreas comunes, los cuales registran continuamente variables ambientales como temperatura, humedad, niveles de CO_2 y luminosidad. Centralizar esta información es fundamental para monitorear las condiciones de los campus, generar alertas tempranas y apoyar la toma de decisiones administrativas y operativas.

Desafíos

El principal desafío de este ecosistema radica en el alto volumen, la velocidad de generación y, sobre todo, la variedad de los datos. Los dispositivos provienen de estructuras de información heterogéneas que evolucionan constantemente. Gestionar esta variabilidad en un esquema relacional resultaría ineficiente, lo que hace indispensable adoptar un modelo flexible que escale junto con la infraestructura de hardware.

Objetivo general

El objetivo de este proyecto es diseñar, implementar y validar una solución de base de datos NoSQL en MongoDB que permita centralizar, gestionar y consultar eficientemente los datos generados por los sensores del campus inteligente. Este desarrollo se enmarca y responde directamente al resultado de aprendizaje de la unidad, aplicando los conceptos fundamentales de las bases de datos no estructuradas para abordar distintas problemáticas y desarrollar soluciones efectivas.

Fundamentación teórica

Comparación entre el enfoque relacional y NoSQL

El uso de un modelo relacional tradicional SQL para este caso presenta limitaciones significativas debido a la rigidez de sus esquemas. Los dispositivos IoT del campus generan datos con estructuras mutables (nuevos sensores añaden nuevos atributos) y a altas frecuencias. En un enfoque relacional esto obligaría a realizar costosas operaciones de ALTER TABLE o a normalizar excesivamente los datos, degradando el rendimiento de escritura (Fowler & Sadalage, 2012). En contraste MongoDB, al ser una base de datos orientada a los documentos, ofrece un esquema dinámico (Schema-less) que permite almacenar mediciones heterogéneas dentro de la misma colección, facilitando la ingesta rápida de datos y adaptándose a la evolución del hardware del campus (Bradshaw et al., 2019).

Desde la perspectiva del teorema CAP (Consistencia, Disponibilidad y Tolerancia a particiones), un sistema de monitoreo distribuido debe priorizar la tolerancia a particiones (P) con su arquitectura de replica sets (Meier &Kaufmann, 2019). Sin embargo, para este ecosistema IoT donde la disponibilidad para escribir mediciones es crítica, la arquitectura de MongoDB Atlas permite ajustar las preferencias de lectura y escritura, balanceando hacia una alta disponibilidad y asegurando que las alertas de CO_2 o temperatura no se pierdan ante fallos parciales del sistema.

Característica	SQL	NoSQL
Estructura de datos	Tablas interconectadas mediante filas y columnas con relaciones estrictas.	Documentos, clave-valor, grafos o familias de columnas.
Diseño del esquema	Rígido: Requiere definir la estructura antes de insertar datos.	Flexible: Permite estructuras heterogéneas.
Escalabilidad	Vertical	Horizontal
Propiedades y CAP	Enfoque estricto en ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad)	Enfoque en el modelo BASE y tolerante a particiones (Teorema CAP), priorizando disponibilidad y rendimiento.
Casos de usos ideales	Sistemas contables, ERPs, inventarios tradicionales y datos altamente normalizados.	Ecosistemas IoT, Big Data, análisis en tiempo real y catálogos con constante evolución de atributos.

Tabla N°1. *Diferencia entre NoSQL y SQL*

Diseño del Esquema de Datos

Descripción del modelo propuesto

El modelo se organiza en cuatro colecciones:

- La colección espacios representa las salas, laboratorios y áreas comunes del campus, y actúa como el punto de partida de toda la topología IoT.
- La colección dispositivos representa los sensores IoT y sus capacidades, y cada documento guarda la referencia al espacio que lo alberga.
- La colección mediciones almacena cada lectura ambiental como un documento independiente, con una marca de tiempo y un arreglo de variables medidas.
- La colección alertas registra los eventos que se generan cuando una lectura supera los umbrales configurados para su espacio, y conserva una copia del valor medido y del umbral para que el evento mantenga su contexto histórico, aunque la medición original cambie.

El modelo prioriza los patrones de acceso del caso, que son la consulta de dispositivos por espacio y la consulta de mediciones por dispositivo y fecha y la consulta de alertas por estado y severidad. Esta estructura permite que el dashboard consulte el estado actual del campus sin recorrer colecciones enteras, y que la ingesta de datos inserte nuevas mediciones sin bloquear lecturas en curso

Colecciones principales

La colección espacio contiene nombre, tipo, capacidad y edificio como campos obligatorios. Además, incluye campos opciones que aportan heterogeneidad, como ubicación, responsable, horario y umbrales. La ubicación se almacena como un objeto GeoJSON con tipo Point y coordenadas, lo que permite consultas espaciales. El responsable se almacena como un objeto anidado con nombre, correo y teléfono, dado que es una entidad dependiente del espacio y no se consulta de forma independiente.
El horario se almacena como un objeto con apertura, cierre y días, porque describe las reglas de operación del espacio. Los umbrales se almacenan como un objeto con una clave por variable y variables min y max, y se usan para evaluar cada lectura durante la ingesta.
La colección dispositiva contiene id_dispositivo, tipo_sensor, id_espacio y estado como campos obligatorios. Además, incluye especificaciones y etiquetas como campos opcionales. El campo tipo_sensor es un arreglo de cadenas que describe las capacidades del sensor, por ejemplo, temperatura y humedad en un mismo dispositivo. Las especificaciones varían según el tipo de sensor, lo que demuestra la heterogeneidad del caso. Las etiquetas permiten agrupar dispositivos por criterios libres, como calidad de aire o aula.
La colección mediciones contiene id_dispositivo, fecha_hora y lecturas. El campo id_medición se usa como identificador legible y único para evitar duplicados en las recargas. El campo lecturas es un arreglo de objetos con tipo, valor y unidad, y cada objeto representa una variable medida en ese instante.

La colección alertas contiene id_espacio, id_dispositivo e id_medición como referencias, además de tipo, severidad, estado, fecha_generación, fecha_resolución, mensaje, valor_registrado y umbral_superado.
Su validador exige las referencias, el tipo de alerta, la severidad, el estado, la fecha de generación y el mensaje. El campo tipo es un enum con siete valores que cubren altas y bajas temperatura, humedad, CO2 y luminosidad. El campo severidad es un enum con cuatro niveles, y el estado es un enum con tres valores que representan el ciclo de vida de la alerta. El campo umbral_superado guarda una copia del umbral min y max que se superó, y valor_registrado gurda el valor que disparó la alerta.

Relaciones entre documentos y cardinalidades relevantes

Relación	Tipo	Decisión
Espacio a Dispositivos	1 a Muchos	Referencia (id_espacio en dispositivo)
Dispositivos a Mediciones	1 a Muchos	Referencia (id_dispositivo en medición)
Espacio a Mediciones	1 a Muchos	Referencia desnormalizada (d_espacio en medición)
Medición a Alertas	1 a Muchos	Referencia (id_medicion en alerta)
Espacio a Alertas	1 a Muchos	Referencia (id_espacio en alerta)
Dispositivos a Alertas	1 a Muchos	Referencia con id_dispositivo en la alerta
Espacio a Umbrales	1 a 1	Embebido
Espacio a ubicación, responsable y horario	1 a 1	Embebido
Dispositivo a especificaciones	1 a 1	Embebido
Mediciones a lecturas	1 a muchos	Embebido

[Figura 2. Tabla de decisiones de embebido y referenciado]

Justificación técnica de las decisiones de embebido y referencias

Se embebe cuando el dato es acotado, se lee junto con el documento principal y no tiene vida propia. Se referencia cuando el dato crece sin límite, se consulta por separado se comparte entre documentos. En este modelo se embeben tipo_sensor y lecturas, y se referencian id_espacio e id_dispositivo.

El campo tipo_sensor se embebe en el dispositivo porque es una lista acotada de capacidades el sensor, se lee junto con el dispositivo y no necesita una colección propia. El campo lecturas se embebe en la medición porque cada lectura pertenece a una medición, la cantidad de variables es acotad y se consulta junto con la marca de tiempo. Un arreglo de objetos con tipo, valor y unidad permite que una medición de temperatura, una de CO2 y una estación ambiental convivan en l misma colección.

El campo id_ aspacio se referencia desde el dispositivo porque un dispositivo puede cambiar de espacio, tiene vida propia y un espacio puede acumular muchos dispositivos. Embeber el arreglo de dispositivos en el espacio haría crecer el documento sin control. El campo id_ dispositivo se referencia desde la medición porque las mediciones son el dato de mayor crecimiento. Un dispositivo puede generar miles de documentos y embeberlos superaría el límite de 16MB por documento.

Los campos adicionales como ubicación, responsable, horario y umbrales en dispositivos se almacenan como atributos heterogéneos. Los validadores actuales no restringen propiedades adicionales, por lo que cada documento puede incluir los atributos que correspondan a su tipo sin modificar el esquema base.

Esta flexibilidad permite que un espacio de tipo Biblioteca incluya horario de atención y uno de tipo Laboratorio incluya umbrales ambientales más estrictos, sin que el validador rechace el documento.

Representación JSON

- **Representación JSON de al menos dos documentos significativos del modelo.**

Documento de Espacios:

```
{
  "_id": "65f1c0a1b2c3d4e5f6a7b8c9",
  "nombre": "Laboratorio de Redes",
  "tipo": "Laboratorio",
  "capacidad": 30,
  "edificio": "Edificio A",
  "ubicacion": {
    "type": "Point",
    "coordinates": [-70.6125, -33.4516]
  },
  "responsable": {
    "nombre": "Diego Fuentes",
    "correo": "diego.fuentes@universidad.cl",
    "telefono": "+56 9 3456 7890"
  },
   "horario": {
     "apertura": "09:00",
     "cierre": "18:00",
     "dias": ["lun", "mar", "mie", "jue", "vie"]
   },
   "umbrales": {
     "temperatura_c": { "min": 16, "max": 28 },
     "humedad_pct": { "min": 30, "max": 70 },
     "co2_ppm": { "max": 800 },
     "luminosidad_lux": { "min": 400, "max": 1000 }
   }
 }
```


Documento de dispositivos:

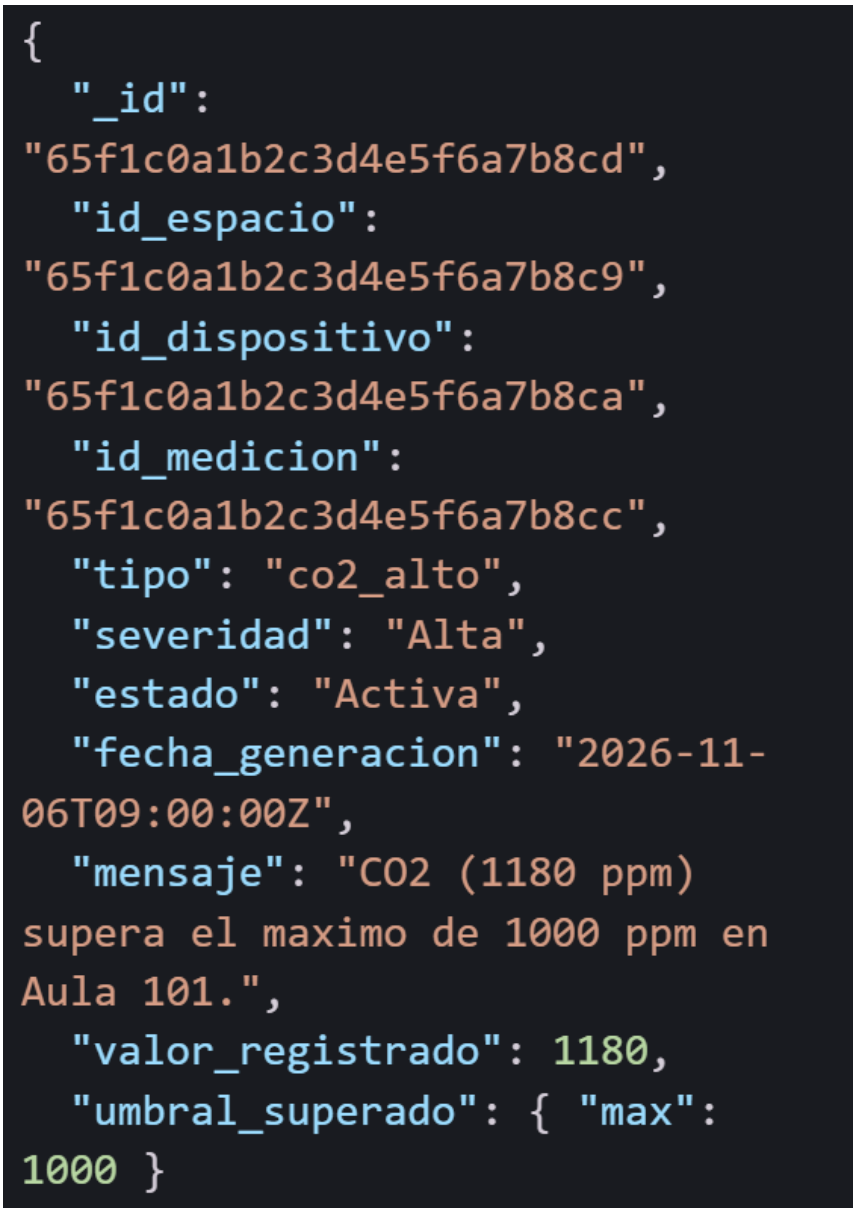
```
{
  "_id": "65f1c0a1b2c3d4e5f6a7b8ca",
  "id_dispositivo": "SEN-CO2-001",
  "tipo_sensor": ["co2", "temperatura"],
  "id_espacio": "65f1c0a1b2c3d4e5f6a7b8c9",
  "estado": "Activo",
  "especificaciones": {
    "modelo": "MH-Z19C",
    "fabricante": "SenseLab",
    "protocolo": "MQTT",
    "voltaje": 5.0,
    "firmware": "v1.4.2",
    "precision": 50
  },
  "etiquetas": ["co2", "aula", "calidad_aire"]
}
```


Documento de mediciones:



```
{
  "_id":
"65f1c0a1b2c3d4e5f6a7b8cc",
  "id_medicion": "MED-001",
  "id_dispositivo":
"65f1c0a1b2c3d4e5f6a7b8ca",
  "fecha_hora": "2026-11-
06T09:00:00Z",
  "lecturas": [
    { "tipo": "temperatura",
"valor": 22.4, "unidad": "C" },
    { "tipo": "humedad", "valor":
48, "unidad": "%" }
  ]
}
```

Documento de alertas:



```
{
  "_id":
"65f1c0a1b2c3d4e5f6a7b8cd",
  "id_espacio":
"65f1c0a1b2c3d4e5f6a7b8c9",
  "id_dispositivo":
"65f1c0a1b2c3d4e5f6a7b8ca",
  "id_medicion":
"65f1c0a1b2c3d4e5f6a7b8cc",
  "tipo": "co2_alto",
  "severidad": "Alta",
  "estado": "Activa",
  "fecha_generacion": "2026-11-
06T09:00:00Z",
  "mensaje": "CO2 (1180 ppm)
supera el maximo de 1000 ppm en
Aula 101.",
  "valor_registrado": 1180,
  "umbral_superado": { "max":
1000 }
```

Las fechas y los identificadores se muestran como cadenas de texto para que el ejemplo sea legible y valido como JSON. En el código se usan ObjectId y datetime. El campo _id es un ObjectId que MongoDB genera al insertar, los campos id_espacio, id_dispositivo e id_medición también son ObjectId en la base, aunque aquí se muestran como cadenas hexadecimales para que el ejemplo sea legible. Los campos id_medición y id_dispositivo son identificadores legibles que se definieron en el catálogo, y se usan para evitar duplicados en las recargas. La diferencia entre un ObjectId y un identificador legible radica en que el primero lo genera la base y el segundo se define en el catálogo de datos.

Explicación del modelo

El índice único sobre espacios.nombre permite buscar un espacio por su nombre y garantiza que no existan dos espacios con el mismo nombre. El índice único sobre dispositivos.id_dispositivo permite recuperar un sensor por su código. El índice parcial sobre mediciones.id_medicion evita duplicados cuando el campo existe, pero permite documentos sin id_medicion, como los de prueba.

El índice compuesto sobre alertas.estado, alertas.severidad y alertas.fecha_generación optimiza la consulta más frecuente del dashboard, qué es l lista de alertas activas ordenadas por severidad y fecha. El índice sobre alertas.espacio_id y alertas.fecha_generacion optimiza la consulta de alertas por espacio. El índice sobre alertas.dispositivo_id y alertas.fecha_generacion optimiza la consulta de alertas por dispositivo. Para las consultas de por espacio, se debería agregar un índice sobre dispositivos.id_espacio y, si se necesita más velocidad, denormalizar id_espacio en mediciones. Es desnormalizción es una mejora posible dado que el modelo actual no la posee, y consiste en copiar el id_espacio en cada medición para evitar la consulta intermedia por dispositivo.

Diagrama del modelo

El diagrama muestra las cuatro colecciones, los campos embebidos y las referencias.

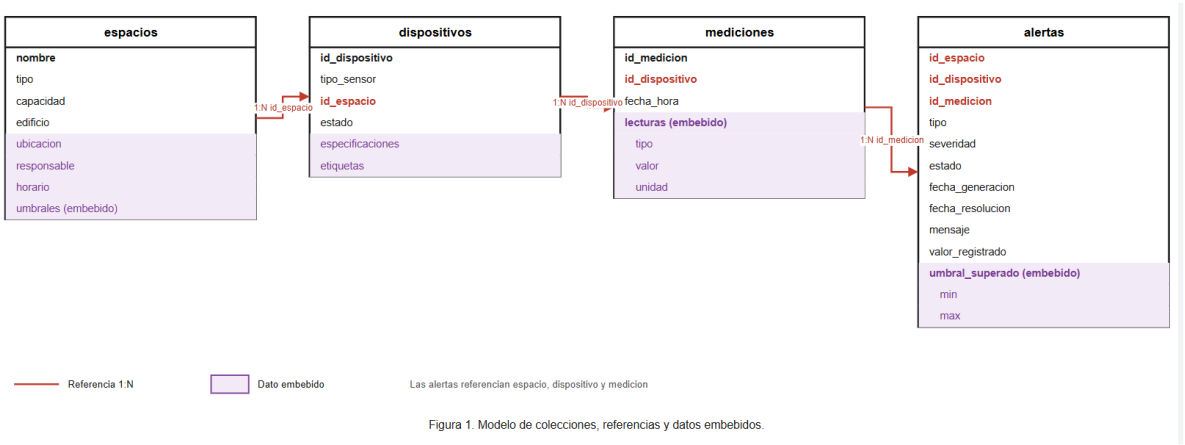


Figura 1. Modelo de colecciones, referencias y datos embebidos.

Inserción de Datos y Operaciones CRUD

Evidencia de código

Archivo:

- insert_one e insert_many sobre datos del caso.

```python
# Demostración de insert_one (primer registro) e insert_many (resto)
if espacios_nuevos:
    resultado_uno = espacios.insert_one(espacios_nuevos[0])
    print(f"insert_one: espacio '{espacios_nuevos[0]['nombre']}' insertado con id {resultado_uno.inserted_id}.")
    espacios_nuevos = espacios_nuevos[1:]
else:
    print("insert_one: no hay espacios nuevos para insertar.")

if espacios_nuevos:
    resultado_masivo = espacios.insert_many(espacios_nuevos)
    print(f"insert_many: {len(resultado_masivo.inserted_ids)} espacios insertados.")
else:
    print("insert_many: no hay espacios nuevos para insertar.")

# Mantener atributos al día mediante upsert
for espacio in sample_data["espacios"]:
    espacios.update_one(
        {"nombre": espacio["nombre"]},
        {"$set": espacio},
        upsert=True,
    )
```

```
PS C:\Users\benja\Solemne02-DB> py src/02_seed_data.py
Conexión exitosa a MongoDB Atlas.
insert_one: espacio 'Aula 101' insertado con id 6af12cd90920d452f7fd60d9.
insert_many: 11 espacios insertados.
insert_one: dispositivo 'SEN-TH-001' insertado con id 6af12cdc0920d452f7fd60e5.
insert_many: 23 dispositivos insertados.
insert_one: medicion 'MED-001' insertada.
insert_many: 4 mediciones insertadas.
insert_many: 1 alerta(s) generada(s) por umbrales superados.

Carga completada:
  - Espacios:     12.
  - Dispositivos: 24.
  - Mediciones:   5.
  - Alertas:      1.
```

- find con filtros aplicados a espacios, dispositivos, mediciones o alertas.

```python
# Recupera una muestra reciente sin traer el identificador interno de MongoDB.
recientes = list(
    mediciones.find(
        {},
        {"_id": 0, "id_medicion": 1, "fecha_hora": 1, "lecturas": 1},
    )
    .sort("fecha_hora", -1)
    .limit(3)
)
print(f"find: se encontraron {len(recientes)} mediciones recientes.")
for medicion in recientes:
    print(medicion)
```

```
find: se encontraron 3 mediciones recientes.
{'id_medicion': 'MED-DEMO-DELETE', 'fecha_hora': datetime.datetime(2026, 11, 6, 9, 20), 'lecturas': [{'tipo': 'luminosidad', 'valor': 410, 'unidad': 'lu
x'}]}
{'id_medicion': 'MED-004', 'fecha_hora': datetime.datetime(2026, 11, 6, 9, 15), 'lecturas': [{'tipo': 'temperatura', 'valor': 23.0, 'unidad': 'C'}, {'ti
po': 'humedad', 'valor': 51, 'unidad': '%'}]}
{'id_medicion': 'MED-003', 'fecha_hora': datetime.datetime(2026, 11, 6, 9, 10), 'lecturas': [{'tipo': 'luminosidad', 'valor': 360, 'unidad': 'lux'}]}
```

- update_one y/o update_many utilizando al menos dos operadores de actualización pertinentes.

```python
# Simula el cambio de estado de un sensor que requiere mantenimiento.
resultado_estado = dispositivos.update_one(
    {"id_dispositivo": "SEN-TEMP-101"},
    {
        "$set": {"estado": "Mantenimiento"},
        "$currentDate": {"actualizado_en": True},
    },
)
print(
    "update_one: "
    f"coincidencias={resultado_estado.matched_count}, "
    f"modificados={resultado_estado.modified_count}."
)
```

```
update_one: coincidencias=1, modificados=1.
```

```
# Marca las mediciones con CO2 sobre 1000 ppm para que sean revisadas.
resultado_revision = mediciones.update_many(
    {
        "lecturas": {
            "$elemMatch": {"tipo": "co2", "valor": {"$gt": 1000}}
        }
    },
    {
        "$set": {"revision_requerida": True},
        "$currentDate": {"marcada_revision_en": True},
    },
)
print(
    "update_many: "
    f"coincidencias={resultado_revision.matched_count}, "
    f"modificadas={resultado_revision.modified_count}."
)
```

```
update_many: coincidencias=1, modificadas=1.
```

- delete_one o delete_many aplicado de forma justificada y controlada.

```
# El filtro limita la eliminacion a un registro temporal del conjunto de prueba.
resultado_eliminacion = mediciones.delete_one(
    {
        "id_medicion": "MED-DEMO-DELETE",
        "etiqueta": "registro_temporal_demo",
    }
)
print(
    "delete_one: "
    f"mediciones de demostracion eliminadas={resultado_eliminacion.deleted_count}."
)
```

```
delete_one: mediciones de demostracion eliminadas=1.
```

4.2.1 Consultas de lectura

Consultas Avanzadas

Filtros

- **Filtros compuestos aplicados al caso mediante $and, $or y $not.**

```
print("DISPOSITIVOS: ")
disp = db["dispositivos"].find(
    {"estado":{"$not": {"$eq":"Activo"}}},
    {'id_dispositivo':1, 'estado':1,'tipo_sensor':1, "_id":0}
    )
for i in disp:
    print(i)
```

```
DISPOSITIVOS:
{'id_dispositivo': 'SEN-TEMP-101', 'estado': 'Mantenimiento', 'tipo_sensor': ['temperatura', 'humedad']}
{'id_dispositivo': 'SEN-LUX-001', 'tipo_sensor': ['luminosidad'], 'estado': 'Mantenimiento'}
{'id_dispositivo': 'SEN-LUX-003', 'tipo_sensor': ['luminosidad'], 'estado': 'Inactivo'}
{'id_dispositivo': 'SEN-CA-002', 'tipo_sensor': ['pm25', 'pm10'], 'estado': 'Mantenimiento'}
```

Operadores

- **Operadores relacionales $gt, $lt y $eq sobre atributos o mediciones.**

```
print("ESPACIOS: ")
espacios = db["espacios"].find(
    {"capacidad": {"$gt": 30}},
    {"nombre":1,"capacidad":1,"_id":0,"tipo":1,"edificio":1}
    )
for i in espacios:
    print(i)
```

```
ESPACIOS:
{'nombre': 'Sala A-101', 'capacidad': 40, 'edificio': 'Edificio A', 'tipo': 'Aula'}
{'nombre': 'Biblioteca Central', 'capacidad': 120, 'edificio': 'Edificio Central', 'tipo': 'Biblioteca'}
{'nombre': 'Aula 101', 'tipo': 'Aula', 'capacidad': 40, 'edificio': 'Edificio A'}
{'nombre': 'Aula 201', 'tipo': 'Aula', 'capacidad': 40, 'edificio': 'Edificio A'}
{'nombre': 'Sala de Estudio 201', 'tipo': 'Espacio Común', 'capacidad': 60, 'edificio': 'Edificio Central'}
{'nombre': 'Auditorio Principal', 'tipo': 'Espacio Común', 'capacidad': 150, 'edificio': 'Edificio D'}
{'nombre': 'Aula 102', 'tipo': 'Aula', 'capacidad': 35, 'edificio': 'Edificio A'}
{'nombre': 'Centro de Computo', 'tipo': 'Espacio Común', 'capacidad': 40, 'edificio': 'Edificio Central'}
```

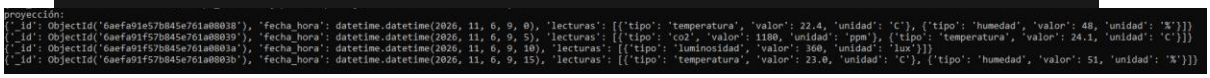
- **Una consulta con $regex aplicada a un campo textual pertinente.**

```
# espacios que sean un laboratorio (con regex)
print("ESPACIOS: ")
espacios = db["espacios"].find(
    {"tipo": {"$regex":"[Ll]aboratorio"}},
    {"nombre":1,"capacidad":1,"_id":0,"tipo":1,"edificio":1}
    )
for i in espacios:
    print(i)
```

```
ESPACIOS:
{'nombre': 'Laboratorio IoT', 'capacidad': 24, 'edificio': 'Edificio B', 'tipo': 'Laboratorio'}
{'nombre': 'Laboratorio de Redes', 'tipo': 'Laboratorio', 'capacidad': 30, 'edificio': 'Edificio A'}
{'nombre': 'Laboratorio de Electronica', 'tipo': 'Laboratorio', 'capacidad': 20, 'edificio': 'Edificio B'}
{'nombre': 'Taller de Mecatronica', 'tipo': 'Laboratorio', 'capacidad': 18, 'edificio': 'Edificio B'}
```

- **Proyecciones y ordenamientos que reduzcan o prioricen la información recuperada.**

```
# proyección básica
print("proyección:")
medi = db["mediciones"].find({}, ["fecha_hora", "lecturas"])
for i in medi:
    print(i)
```



Agregaciones

- **Un pipeline que utilice $match + $group.**

```
# promedio y maximo de cada variable ambiental ($match + $group)
print("PROMEDIO POR VARIABLE: ")
mediciones = db["mediciones"].aggregate([
    {"$unwind": "$lecturas"},
    {"$match": {"lecturas.tipo": {"$in": ["temperatura", "humedad", "co2", "luminosidad"]}}},
    {"$group": {"_id": "$lecturas.tipo", "promedio": {"$avg": "$lecturas.valor"}, "maximo": {"$max": "$lecturas.valor"}}}
    ])
for i in mediciones:
    print(i)
```

```
PROMEDIO POR VARIABLE:
{'_id': 'luminosidad', 'promedio': 360.0, 'maximo': 360}
{'_id': 'co2', 'promedio': 1180.0, 'maximo': 1180}
{'_id': 'temperatura', 'promedio': 23.166666666666668, 'maximo': 24.1}
{'_id': 'humedad', 'promedio': 49.5, 'maximo': 51}
```

Explicación: Se separan las lecturas en cada medición, se filtran las variables del caso y se calcula su promedio y máximo. Permite detectar rápidamente si alguna variable, como el CO2, supera niveles adecuados.

- **Un pipeline que utilice $project e incluya al menos un campo calculado o transformado.**

```python
# cantidad de variables que mide cada dispositivo activo ($project con campo calculado)
print("VARIABLES POR DISPOSITIVO: ")
mide = db["dispositivos"].aggregate([
    {"$match": {"estado": "Activo"}},
    {"$project": {"_id": 0, "id_dispositivo": 1, "cantidad_variables": {"$size": "$tipo_sensor"}}}
    ])
for i in mide:
    print(i)
```

```
VARIABLES POR DISPOSITIVO:
{'id_dispositivo': 'SEN-CO2-LAB', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-LUX-BIB', 'cantidad_variables': 1}
{'id_dispositivo': 'SEN-TH-001', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-CO2-001', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-TH-002', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-EA-001', 'cantidad_variables': 4}
{'id_dispositivo': 'SEN-CA-001', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-TH-003', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-CO2-002', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-TH-004', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-LUX-002', 'cantidad_variables': 1}
{'id_dispositivo': 'SEN-TH-005', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-CO2-003', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-EA-002', 'cantidad_variables': 4}
{'id_dispositivo': 'SEN-TH-006', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-CO2-004', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-TH-007', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-TH-008', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-LUX-004', 'cantidad_variables': 1}
{'id_dispositivo': 'SEN-TH-009', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-CO2-005', 'cantidad_variables': 2}
{'id_dispositivo': 'SEN-EA-003', 'cantidad_variables': 4}
{'id_dispositivo': 'SEN-CA-003', 'cantidad_variables': 2}
```

Explicación: Se muestra cada dispositivo activo con el campo calculado cantidad_variables, *obteniendo con $size* sobre tipo_sensor. Ayuda a priorizar el mantenimiento de los sensores que miden más variables.

- **Un pipeline que utilice $sort y $limit para generar un ranking o resumen.**

```python
# top 3 variables mas monitoreadas ($sort + $limit)
print("TOP 3 VARIABLES: ")
variables = db["dispositivos"].aggregate([
    {"$unwind": "$tipo_sensor"},
    {"$group": {"_id": "$tipo_sensor", "dispositivos": {"$sum": 1}}},
    {"$sort": {"dispositivos": -1}},
    {"$limit": 3}
    ])
for i in variables:
    print(i)
```

```
TOP 3 VARIABLES:
{'_id': 'temperatura', 'dispositivos': 19}
{'_id': 'humedad', 'dispositivos': 13}
{'_id': 'co2', 'dispositivos': 9}
```

Explicación: Se cuenta cuantos dispositivos miden cada variable y se muestran las tres más monitoreadas. Las que no aparecen indican donde conviene instalar nuevos sensores.

Discusión de Resultados

El modelo logrado destaca por su alta flexibilidad, logrando asimilar el registro de todos los dispositivos IoT. Como limitante, la separación estructural entre los espacios y las mediciones referenciadas requieren uso de operadores complejos, lo que incrementa el esfuerzo computacional.

Verificación de ejecución en MongoDB Atlas (pruebas integrales)

Para comprobar que la solución funciona de punta a punta en la nube, se desarrolló el script src/07_integration_tests.py. Este script ejecuta en secuencia todos los módulos del repositorio (validadores, carga de datos, CRUD, consultas y agregaciones) sobre una base de prueba independiente del clúster de MongoDB Atlas (campus_inteligente_test), de modo que las pruebas no alteran los datos de trabajo del equipo. Cada prueba verifica un resultado concreto en la base, no solo que el código no lance errores; registra su tiempo de ejecución y su salida, y al terminar se genera automáticamente un reporte en docs/evidencias/ que respalda los resultados de esta sección.|

La Tabla 3 resume las 15 pruebas ejecutadas y el resultado obtenido en Atlas.

Código	Prueba	Qué se verifica	Resultado
P01	Conexión a Atlas	Que la conexión apunta a un clúster *.mongodb.net y responde al ping.	OK
P02	Colecciones con validadores	Que las 4 colecciones se crean con su $jsonSchema.	OK
P03	Carga inicial	Que los conteos coinciden con el dataset (12 espacios, 24 dispositivos, mediciones y alertas esperadas).	Advertencia (solo 5 mediciones)
P04	Re-ejecución de validadores	Que el script de validadores se puede ejecutar dos veces sin error.	FALLA
P05	Carga idempotente	Que una segunda carga no duplica documentos.	OK
P06	Rechazo de inválidos	Que 8 documentos mal formados (enum, tipos, campos faltantes) son rechazados con el código 121.	OK (8/8)
P07	Flexibilidad del esquema	Que un sensor de un tipo nuevo (ruido, dB) y su lectura se aceptan sin cambiar el esquema.	OK
P08	Integridad referencial	Que no hay referencias huérfanas entre las 4 colecciones (verificado con $lookup).	OK (0 huérfanos)
P09	Coherencia de alertas	Que cada alerta supera realmente su umbral y su severidad coincide con el % de desvío.	OK
P10	Operaciones CRUD	Que update_one, update_many y delete_one producen el efecto esperado en la base.	FALLA
P11	Integridad tras CRUD	Que las modificaciones y eliminaciones no dejan referencias rotas.	OK
P12	Consultas de filtrado	Que 04_query_filters.py se ejecuta sin errores en Atlas.	OK
P13	Pipelines de agregación	Que 05_aggregations.py se ejecuta sin errores en Atlas.	OK
P14	Cobertura de la rúbrica	Que el código incluye todos los operadores exigidos ($and, $or, $not, $gt, $lt, $eq,	Advertencia (faltan

		$regex, $match, $group, $project, $sort, $limit).	$and, $or, $lt)
P15	Índices y explain	Que la consulta de alertas activas usa el índice compuesto (IXSCAN).	OK (IXSCAN)

Tabla 3 . Batería de pruebas integrales ejecutadas en MongoDB Atlas.

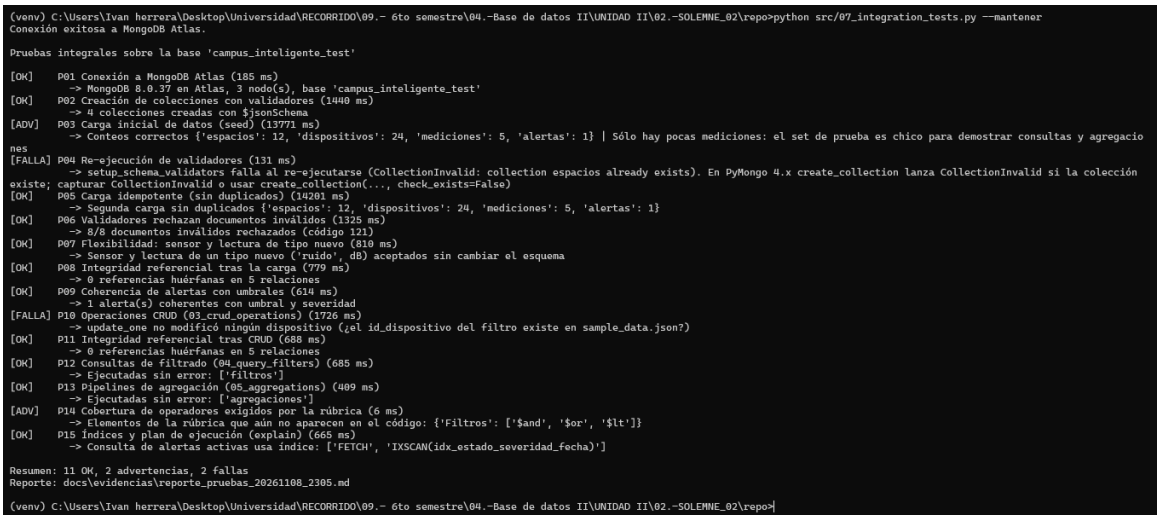


Figura 15. Ejecución de las pruebas integrales sobre MongoDB Atlas (salida de python src/07_integration_tests.py).

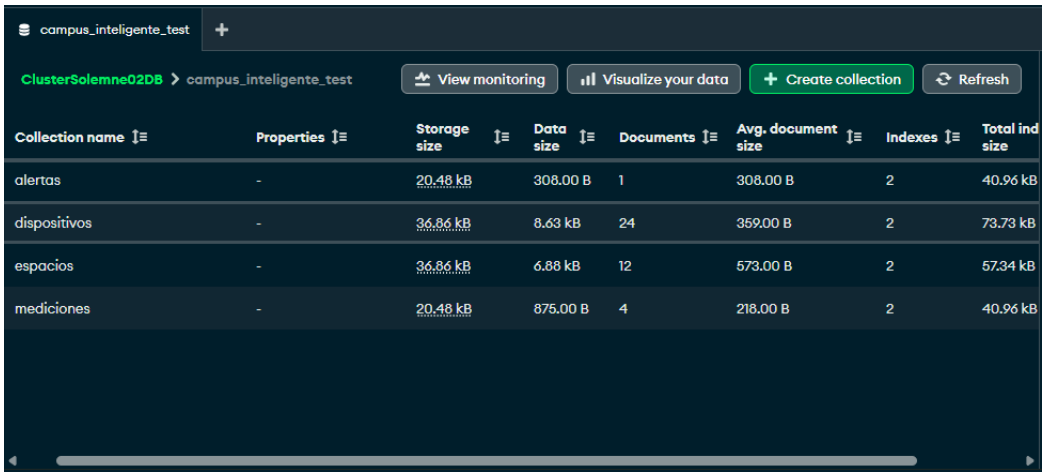


Figura 16. Base campus_inteligente_test en el Data Explorer de Atlas tras las pruebas: 12 espacios, 24 dispositivos, 4 mediciones y 1 alerta.

Resultados. La ejecución en Atlas arrojó **11 pruebas aprobadas, 2 advertencias y 2 fallas**, con un tiempo total de 37,4 segundos (servidor MongoDB 8.0.37, clúster de 3 nodos, ejecución del 08-11-2026 a las 23:05). Además de validar el sistema, las pruebas permitieron detectar dos errores antes de la entrega: (1) el script de validadores no podía volver a ejecutarse, porque en PyMongo 4.x create_collection lanza CollectionInvalid cuando la colección ya existe, y (2) el update_one del script CRUD filtraba por un dispositivo inexistente (SEN-TEMP-101), por lo que no modificaba ningún documento. Las fallas P04 y P10 corresponden exactamente a esos dos errores, que fueron informados a los responsables de cada script. Las advertencias

indican que el dataset de prueba tiene solo 5 mediciones (P03) y que el script de consultas aún no incluye los operadores $and, $or y $lt (P14).

Evaluación crítica del modelo implementado

Fortalezas. El modelo responde bien a la naturaleza heterogénea de los datos del campus. Las mediciones guardan sus valores en el arreglo lecturas, donde cada elemento tiene la forma {tipo, valor, unidad}. Gracias a esto, un termohigrómetro (temperatura y humedad), un sensor de CO_2 y un sensor de material particulado (PM2.5 y PM10) conviven en la misma colección sin columnas vacías. En las pruebas se incorporó un sensor de un tipo inexistente (ruido, en dB), que fue aceptado y consultado sin modificar el esquema ni el código (P07). Los datos que siempre se leen junto al espacio (ubicación GeoJSON, responsable, horario y umbrales) están embebidos, por lo que una sola lectura del espacio entrega todo lo necesario para evaluar sus condiciones. Las colecciones que crecen o cambian de forma independiente (dispositivos, mediciones y alertas) se referencian por ObjectId, lo que evita documentos que crezcan sin límite. Los validadores $jsonSchema agregan una capa de calidad: rechazaron 8 de 8 documentos inválidos (P06) y aun así dejan libres los campos no críticos (MongoDB, Inc., 2026b). Por último, las alertas guardan el umbral superado y el valor registrado en el momento del evento, así que conservan su sentido aunque después cambie el umbral del espacio; la prueba P09 confirmó que todas son coherentes.

Limitaciones.

- *Integridad referencial en manos de la aplicación:* MongoDB no tiene claves foráneas. Si se elimina una medición o un dispositivo, sus alertas pueden quedar huérfanas. En nuestro set no hubo huérfanos (0 en P08 y P11), pero eso lo garantizan los scripts y la verificación con $lookup, no el motor.
- *Consultas más verbosas sobre el arreglo lecturas:* filtrar por una variable exige $elemMatch (por ejemplo, tipo "co2" con valor mayor a 1000) y agregar por variable exige $unwind. Sin un índice multikey sobre lecturas.tipo y lecturas.valor, esas consultas recorren toda la colección.
- *Mediciones sin referencia directa al espacio:* en la implementación actual, para saber las condiciones de un espacio hay que pasar por sus dispositivos ($lookup adicional), tal como se señaló al inicio de esta sección. Esto encarece las consultas por espacio, que son las más frecuentes del panel de monitoreo.
- *Alertas generadas durante la carga:* las alertas se calculan en Python al insertar. Una medición que llegue por otra vía (por ejemplo, desde un gateway IoT directo a Atlas) no generaría alerta, y si se modifica un umbral las mediciones históricas no se reevalúan.
- *Volumen del set de prueba:* el dataset tiene solo 5 mediciones, que generan 1 alerta. Basta para validar el funcionamiento, pero no para medir rendimiento real ni obtener indicadores estadísticamente representativos.

Mejora concreta propuesta. Migrar la colección de mediciones a una **colección time series** de MongoDB (timeField: fecha_hora; metaField: {id_dispositivo, id_espacio}), con expiración automática del dato crudo (expireAfterSeconds) y promedios horarios guardados en una colección de resumen mediante $merge

(MongoDB, Inc., 2026c). MongoDB agrupa y comprime internamente las lecturas por dispositivo y tiempo, lo que reduce almacenamiento e índices sin cambiar las consultas. Incluir id_espacio en el metaField resuelve además la limitación de las consultas por espacio. Como complemento, la generación de alertas se trasladaría a un Atlas Trigger (change stream) que evalúe cada medición en el servidor al momento de insertarse, sin importar qué cliente la envió.

Comparación con una alternativa relacional

Un diseño relacional para el mismo problema usaría tablas de espacios, dispositivos, mediciones y alertas unidas por claves foráneas. El punto difícil serían las mediciones heterogéneas, y hay dos salidas. La primera es una tabla ancha con una columna por variable, que queda llena de valores NULL y obliga a ejecutar ALTER TABLE cada vez que se instala un sensor nuevo. La segunda es una tabla de lecturas (medición, variable, valor), que multiplica las filas y exige JOIN y pivotes para reconstruir una medición. Lo mismo ocurre con las especificaciones técnicas, que cambian según fabricante y modelo (voltaje, precisión, protocolo, firmware). En MongoDB, en cambio, la medición completa es un solo documento y el sensor de ruido se incorporó sin migración (Fowler & Sadalage, 2012).

A cambio, el modelo relacional ofrece integridad referencial y transacciones ACID garantizadas por el motor, además de SQL, un lenguaje más conocido para reportes. Para datos maestros y estables, como el catálogo de espacios, una base relacional sería igual de adecuada. La ventaja de MongoDB se concentra en las colecciones de alto volumen y forma variable (mediciones y alertas), que son el núcleo del sistema de monitoreo (Bradshaw et al., 2019).

Criterio	MongoDB (solución implementada)	Alternativa relacional
Sensor con variables nuevas	Se inserta tal cual (P07)	ALTER TABLE o filas adicionales en una tabla de atributos
Leer una medición completa	1 documento	JOIN / pivote entre tablas
Integridad referencial	Responsabilidad de la aplicación (verificada en P08 y P11)	Claves foráneas del motor
Escalamiento de escrituras	Horizontal (sharding)	Principalmente vertical
Consultas analíticas	Aggregation framework	SQL estándar

Tabla 4. Comparación entre la solución MongoDB y una alternativa relacional.

Escalabilidad, flexibilidad del esquema y eficiencia de las consultas

Escalabilidad. La colección que crece de forma continua es la de mediciones. Con 24 dispositivos que reportan cada 5 minutos se generan unas 6.900 mediciones al día, es decir, cerca de 2,5 millones al año. Atlas permite fragmentar (sharding) esa

colección con una clave como {id_dispositivo, fecha_hora}, que reparte la escritura entre servidores y dirige las consultas de un sensor a un solo fragmento. Las colecciones pequeñas (espacios y dispositivos) no necesitan fragmentarse. El replica set de Atlas permite además ajustar el balance del teorema CAP por operación: las mediciones pueden escribirse con w:1 para privilegiar disponibilidad y latencia, y las alertas con w:"majority" para privilegiar consistencia.

Flexibilidad del esquema. Se comprobó en tres niveles: (a) dispositivos con tipo_sensor y especificaciones distintas dentro de la misma colección; (b) mediciones con distinta cantidad y tipo de lecturas; y (c) un tipo de sensor nuevo aceptado en tiempo de ejecución (P07). Los validadores solo controlan los campos esenciales (referencias, fechas y enumeraciones de estado y severidad), con lo que se equilibran la flexibilidad y la calidad del dato.

Eficiencia de las consultas. Existen índices únicos sobre espacios.nombre, dispositivos.id_dispositivo y mediciones.id_medicion, además de un índice compuesto {estado, severidad, fecha_generacion} en alertas. El explain() de la prueba P15 mostró que la consulta del panel de alertas activas usa ese índice (etapa IXSCAN sobre idx_estado_severidad_fecha) en lugar de recorrer la colección (MongoDB, Inc., 2026a). Las proyecciones de los scripts de consulta reducen los campos transferidos. Como oportunidad de mejora, faltan índices para los dos patrones de acceso más frecuentes sobre mediciones: {id_dispositivo, fecha_hora} (historial de un sensor) y {lecturas.tipo, lecturas.valor} (búsqueda de valores fuera de umbral). Con el volumen proyectado, estos índices evitarían recorridos completos de la colección.

Conclusiones

En este proyecto pudimos enfrentar un contexto acercado a la realidad, en donde tenemos que usar una base de datos NoSQL, lo que nos permite asimilar y aplicar los conocimientos que hemos obtenidos en la Unidad 2 de esta materia para generar sistemas que generen valor, logrando así diseñar un modelo robusto que puede registrar grandes y variados volúmenes de datos en un sólo lugar, de manera centralizada en donde generar consultas. MongoDB representa una gran herramienta para problemas que requieran estructuras flexibles, como lo son los IoT, dándonos con esto conocimiento valioso para futuros proyectos.

Síntesis de aprendizajes. El desarrollo mostró que modelar en NoSQL parte de las consultas que el sistema debe responder y no de la normalización. La decisión de embeber umbrales, horario y responsable en el espacio, y de referenciar dispositivos, mediciones y alertas, se tomó pensando en cómo se leen los datos y en cuánto crecen. También aprendimos que la flexibilidad del esquema no reemplaza el control de calidad: los validadores $jsonSchema, los índices y las verificaciones de integridad en la aplicación fueron necesarios para que el sistema fuera confiable. Por último, las pruebas integrales sobre Atlas permitieron detectar errores reales antes de la entrega, como un script que no se podía re-ejecutar y una actualización que no encontraba su documento. Eso confirma la importancia de validar el flujo completo y no cada script por separado.

Cumplimiento del objetivo del caso 2026. La solución implementada en MongoDB Atlas permite: (1) registrar los espacios del campus y sus dispositivos IoT asociados (12 espacios y 24 dispositivos con atributos heterogéneos); (2) almacenar mediciones ambientales de diferentes sensores mediante el arreglo lecturas; (3) identificar las mediciones que superan los umbrales definidos para cada espacio, generando alertas con severidad calculada; (4) consultar el estado de los dispositivos y las condiciones ambientales mediante filtros compuestos, operadores relacionales, $regex, proyecciones y ordenamientos; y (5) generar indicadores para la toma de decisiones con pipelines de agregación. Todas las operaciones se ejecutaron y verificaron en Atlas (11 de 15 pruebas aprobadas; las 2 fallas y 2 advertencias corresponden a correcciones puntuales ya identificadas), por lo que el objetivo de diseñar, implementar y validar una solución NoSQL para el caso se considera cumplido.

Pertinencia de MongoDB. Para un escenario de monitoreo IoT, con datos semiestructurados, crecimiento continuo y dispositivos que evolucionan, MongoDB es una opción pertinente. Absorbe la variedad de sensores sin migraciones, escala horizontalmente cuando el volumen de mediciones lo exige y permite ajustar la consistencia según la criticidad de cada operación. Su principal costo es trasladar a la aplicación parte de la integridad que en un modelo relacional garantiza el motor. En

este proyecto ese costo se mitigó con validadores de esquema y pruebas automáticas de integridad.

Bibliografía

- Fowler & Sadalage (2012). *NoSQL Distilled*. Addison-Wesley.
- Bradshaw et al. (2019). *MongoDB: The Definitive Guide*. O'Reilly.
- Meier & Kaufmann (2019). Introducción a modelos de datos NoSQL.
- MongoDB, Inc. (2026a). Explain results [Documentación]. MongoDB Manual. https://www.mongodb.com/docs/manual/reference/explain-results/
- MongoDB, Inc. (2026b). Schema validation [Documentación]. MongoDB Manual. https://www.mongodb.com/docs/manual/core/schema-validation/
- MongoDB, Inc. (2026c). Time series collections [Documentación]. MongoDB Manual. https://www.mongodb.com/docs/manual/core/timeseries-collections/

Anexos

Los scripts completos de creación, carga, operaciones, consultas, agregaciones y pruebas, junto con el dataset de prueba y el reporte de ejecución en Atlas, están disponibles en el repositorio del proyecto: https://github.com/c0d3312/Solemne02-DB

- src/01_db_connection.py: conexión a MongoDB Atlas y validadores de esquema ($jsonSchema).
- src/02_seed_data.py: carga de espacios, dispositivos y mediciones, y generación automática de alertas.
- src/03_crud_operations.py: operaciones de lectura, actualización y eliminación.
- src/04_query_filters.py: consultas de filtrado con operadores lógicos, relacionales y $regex.
- src/05_aggregations.py: pipelines de agregación.
- src/06_main_runner.py: orquestador de ejecución.
- src/07_integration_tests.py: pruebas integrales y verificación en MongoDB Atlas.
- data/sample_data.json: dataset de prueba (12 espacios, 24 dispositivos y mediciones).
- docs/evidencias/reporte_pruebas_20261108_2305.md: reporte generado por las pruebas en Atlas.